COMPETITION PROJECT

Smart Person-Tracking Car

Autonomous Color-Based Tracking with ESP32-S3-CAM & Safety System

by:
Anwar Mithal Abdulhussein
Sabreen Hamed Khaleel

Supervised by
Assist. Prof. Dr. Heba Mohammed Fadhil

OpenCV | ESP32 | CamShift | Wi-Fi | US100 Safety

THE PROBLEM

Why Do We Need This System?

1

Manual Control Fatigue

Continuous manual operation is tiring and error-prone — operators cannot maintain safe distance reliably.

2

Collision Risk

Without automated safety, robots can collide with people and obstacles during forward motion.

3

Deep Learning is Expensive

State-of-the-art neural tracking models are too computationally heavy for low-cost ESP32 MCUs.

Goal: Build an autonomous, low-cost tracking robot that is safe, intelligent, and practical.

OUR SOLUTION

Two-Layer Architecture

PC — TrackerApp

VisionModule

OpenCV frame capture & color tracking

ColorTracker

HSV histogram + CamShift algorithm

CommandModule

HTTP client — sends motion commands

HTTP
REST

ESP32-S3-CAM — Firmware

CameraModule

OV2640 — live frame streaming

MotorController

4WD — Forward / Back / Left / Right

DistanceSensor

US100 ultrasonic — collision guard

HARDWARE COMPONENTS

What's Inside the Robot?

ESP32-S3-CAM

Brain + Camera

240 MHz dual-core MCU with integrated OV2640 camera. Runs HTTP server, motor control, and Wi-Fi stack.

4WD DC Motors

Movement

Four-wheel drive chassis for forward, backward, left, and right movement, controlled via PWM signals.

US100 Sensor

Safety Guard

Ultrasonic distance sensor. Triggers emergency stop when obstacle detected within safe distance threshold.

LED + Buzzer

Status Feedback

Visual LED indicator and buzzer provide real-time status and alert the operator on safety override events.

How Does the Robot See and Think?

1 Capture
Camera frame (OV2640)

→

2 Preprocess
RGB → HSV
Blur + Resize

→

3 Detect
HSV histogram + Backproject

→

4 Track
CamShift algorithm

→

5 Act
L / R / FWD / STOP

Why Classical CV, Not Deep Learning?

- ESP32-S3 has ~512 KB RAM
- Neural models require MBs of memory & GPU
- CamShift achieves 85% accuracy at ~150ms/frame

Distance-Based Safety Logic

- US100 reads distance every 40 ms
- If distance < SAFE_DISTANCE_CM → emergency stop
- Buzzer alert activates; overrides all motion commands

How Everything Connects

PC TrackerApp

VisionModule
(OpenCV)

CommandModule
(HTTP Client)

UI / Keyboard
Control

HTTP / REST

/capture /go /left
/right /stop /distance

Wi-Fi

RoverFirmware (ESP32-S3-CAM)

CameraModule

DCAM / OV2640

MotorController

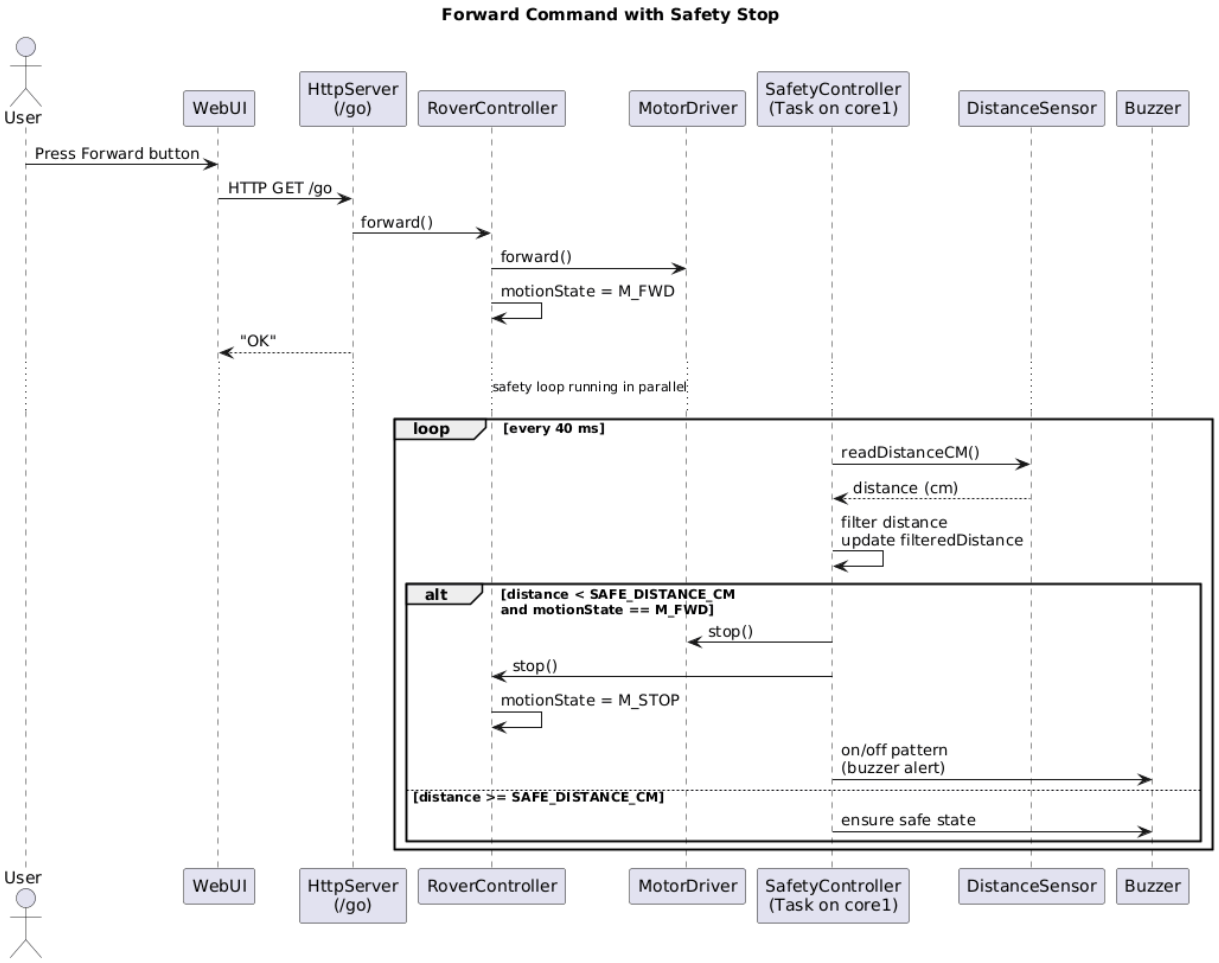
4WD Drive

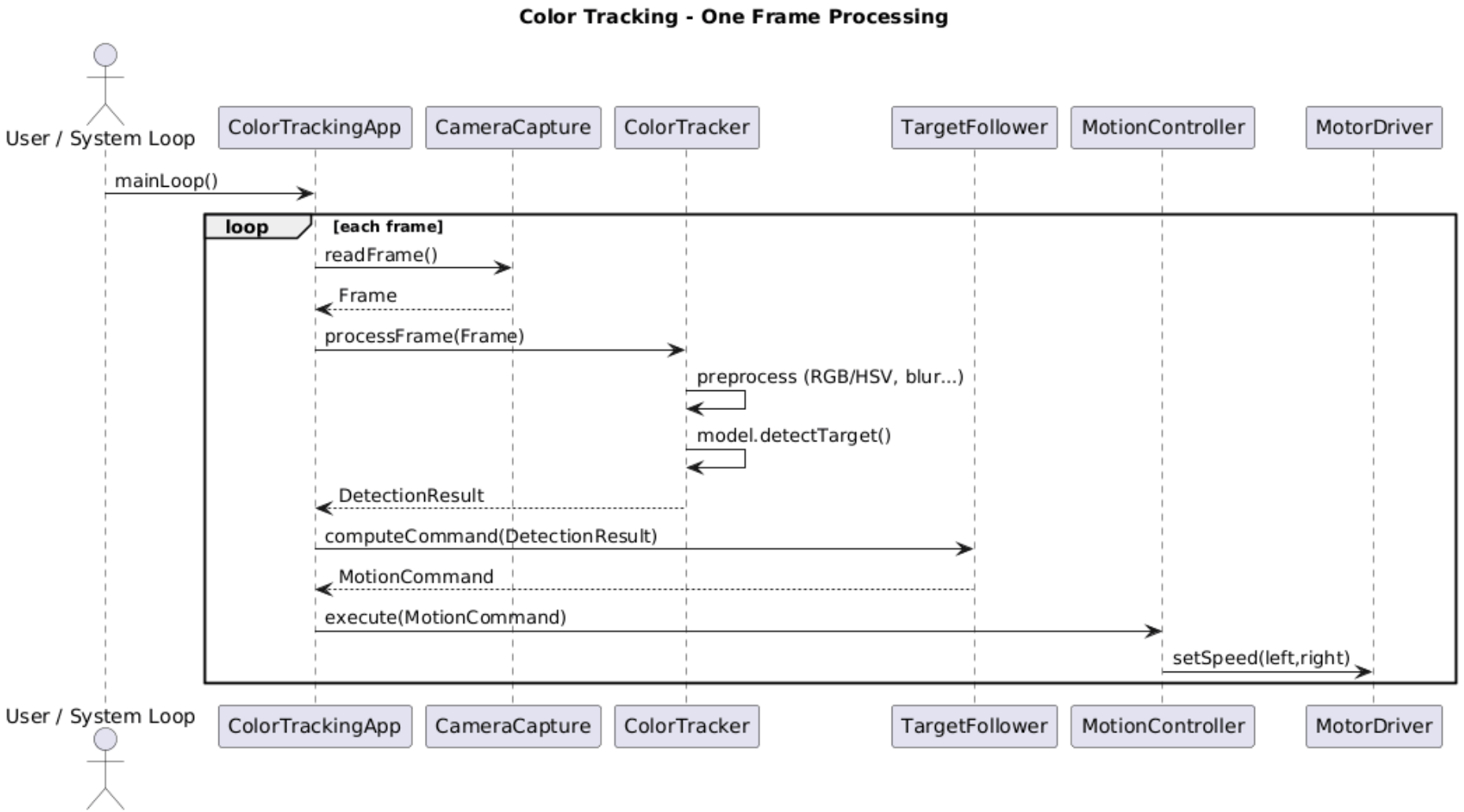
DistanceSensor

US100 Ultrasonic

WebServer

HTTP API


Forward Command with Safety Stop
User
WebUI
HttpServer (/go)
RoverController
MotorDriver
SafetyController (Task on core1)
DistanceSensor
Buzzer
Press Forward button
HTTP GET /go
forward()
forward()
motionState = M_FWD
"OK"
safety loop running in parallel
loop
[every 40 ms]
readDistanceCM()
distance (cm)
filter distance
update filteredDistance
alt
[distance < SAFE_DISTANCE_CM
and motionState == M_FWD]
stop()
stop()
motionState = M_STOP
on/off pattern
(buzzer alert)
[distance >= SAFE_DISTANCE_CM]
ensure safe state


Color Tracking - One Frame Processing
User / System Loop
ColorTrackingApp
CameraCapture
ColorTracker
TargetFollower
MotionController
MotorDriver
mainLoop()
loop
[each frame]
readFrame()
Frame
processFrame(Frame)
preprocess (RGB/HSV, blur...)
model.detectTarget()
DetectionResult
computeCommand(DetectionResult)
MotionCommand
execute(MotionCommand)
setSpeed(left,right)
User / System Loop
ColorTrackingApp
CameraCapture
ColorTracker
TargetFollower
MotionController
MotorDriver

How Well Does It Work?

85%

Tracking Accuracy

Under moderate indoor lighting

~150ms

Per-Frame Processing

120–180 ms range (CamShift)

100%

Safety Reliability

Zero collisions during tests

- Stable person-tracking achieved indoors under moderate lighting conditions
- Distance-based safety system prevented all front collisions during testing
- Performance degrades with sudden lighting changes or similar background colors

Technologies Used

Hardware

- ESP32-S3-CAM — MCU + camera module
- 4WD DC motor chassis
- US100 Ultrasonic distance sensor
- LED indicator + Buzzer

Software

- Arduino IDE — ESP32 firmware
- Python — PC tracking script
- OpenCV — CamShift + backprojection
- Requests — HTTP command sending

Design Architecture | Develop Firmware | Implement Python Tracker | Tune & Test

CONCLUSIONS & FUTURE WORK

✓ What We Achieved

- Low-cost person-tracking car is feasible with ESP32-S3-CAM and classical CV
- Color-based tracking is effective under stable indoor lighting (~85% accuracy)
- Safety system successfully prevents collisions using US100 sensor override
- Solid baseline for future work with more advanced techniques

→ Future Enhancements

1. **YOLOv5-nano Integration**

   Lightweight deep-learning model optimized for edge MCUs

2. **Multi-Person Tracking**

   Extend tracking to handle multiple targets simultaneously

3. **360° Obstacle Avoidance**

   Add sensors on all sides for complete collision protection

4. **MQTT Smart Home Link**

   Connect robot to IoT home network for remote monitoring

Thank You!

Questions & Discussion

85% Accuracy	~150ms Per Frame	4WD Drive System	Wi-Fi Protocol